



Вычислительная геометрия и алгоритмы компьютерной графики

Лекция №3: Графический конвейер. Структура графического приложения

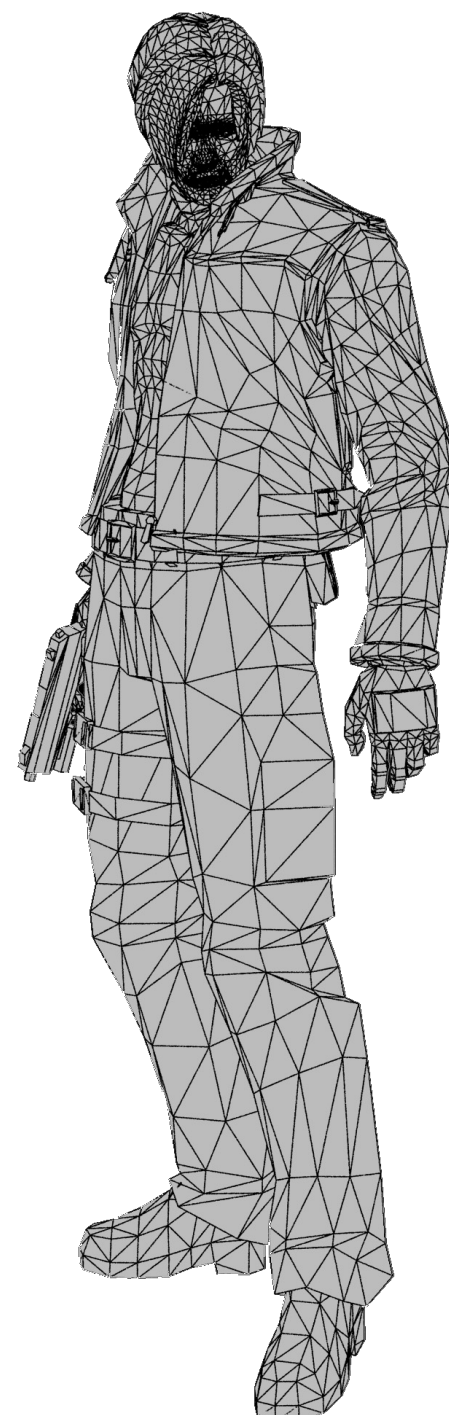
к.ф.-м.н., доцент

Рябинин Константин Валентинович,
kostya.ryabinin@gmail.com

ст. преподаватель

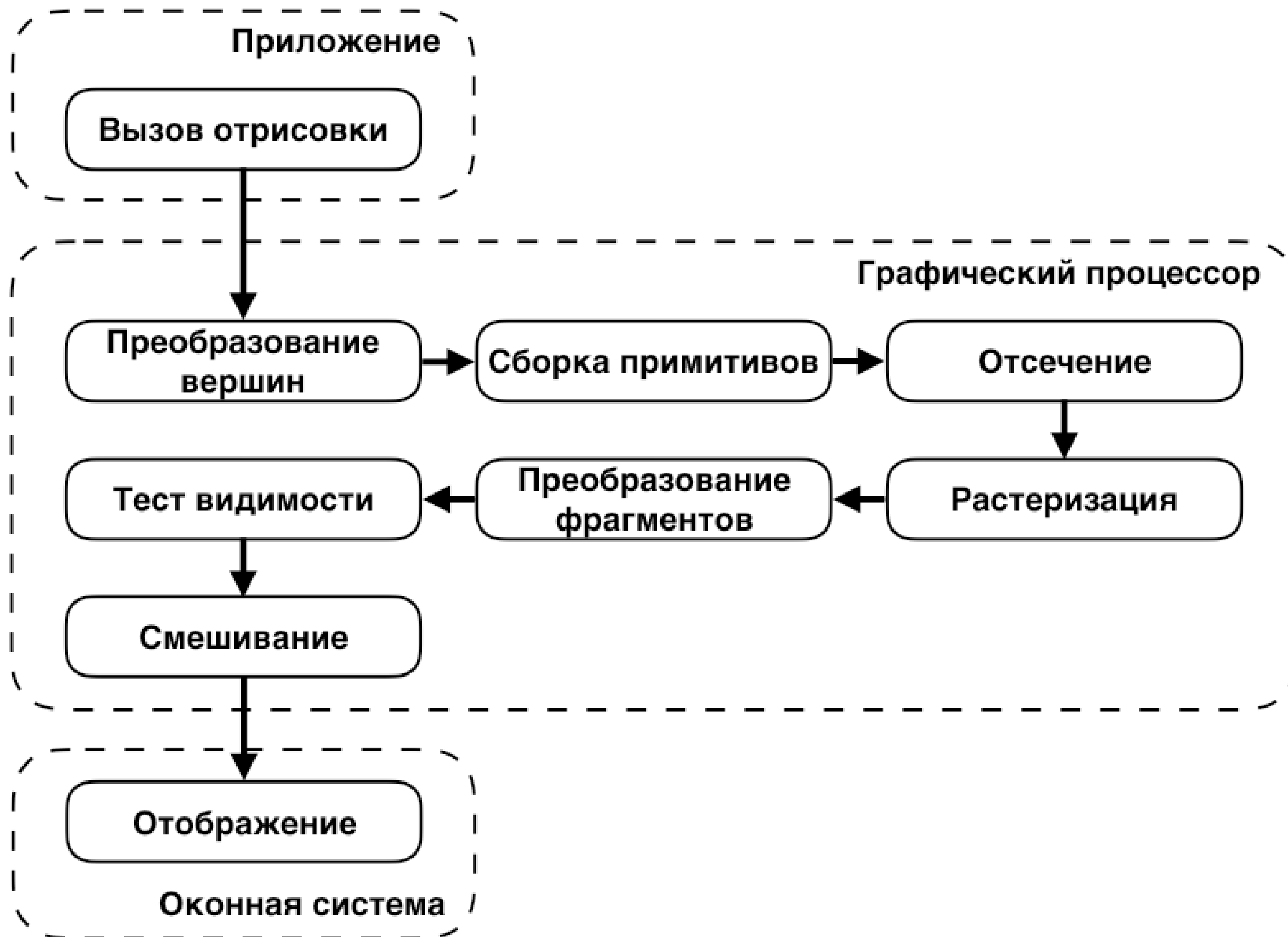
Лабутин Иван Александрович,
linuxrf@gmail.com

Объекты в полигональной 3D-графике представляют собой поверхности, аппроксимированные множеством многоугольников



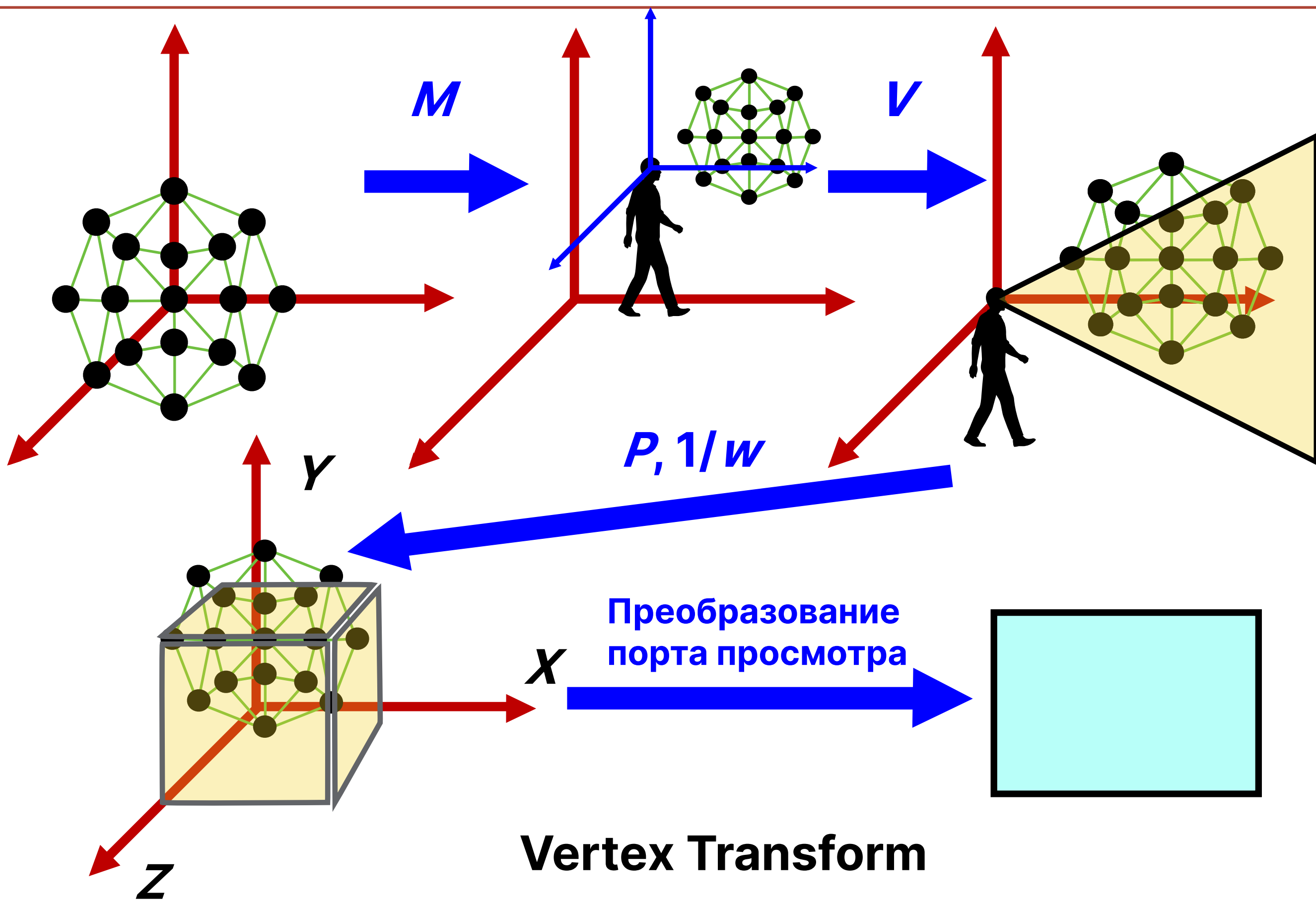
- Атомарная управляемая единица геометрии – **вершина**
- Вершина имеет набор атрибутов (типа float), интерпретация которых, вообще говоря, лежит на программисте:
 - Координаты в пространстве
 - Координаты нормали
 - Координаты текстуры
 - Цвет
 - ...
- Вершины объединяются в примитивы, почти всегда – **треугольники**

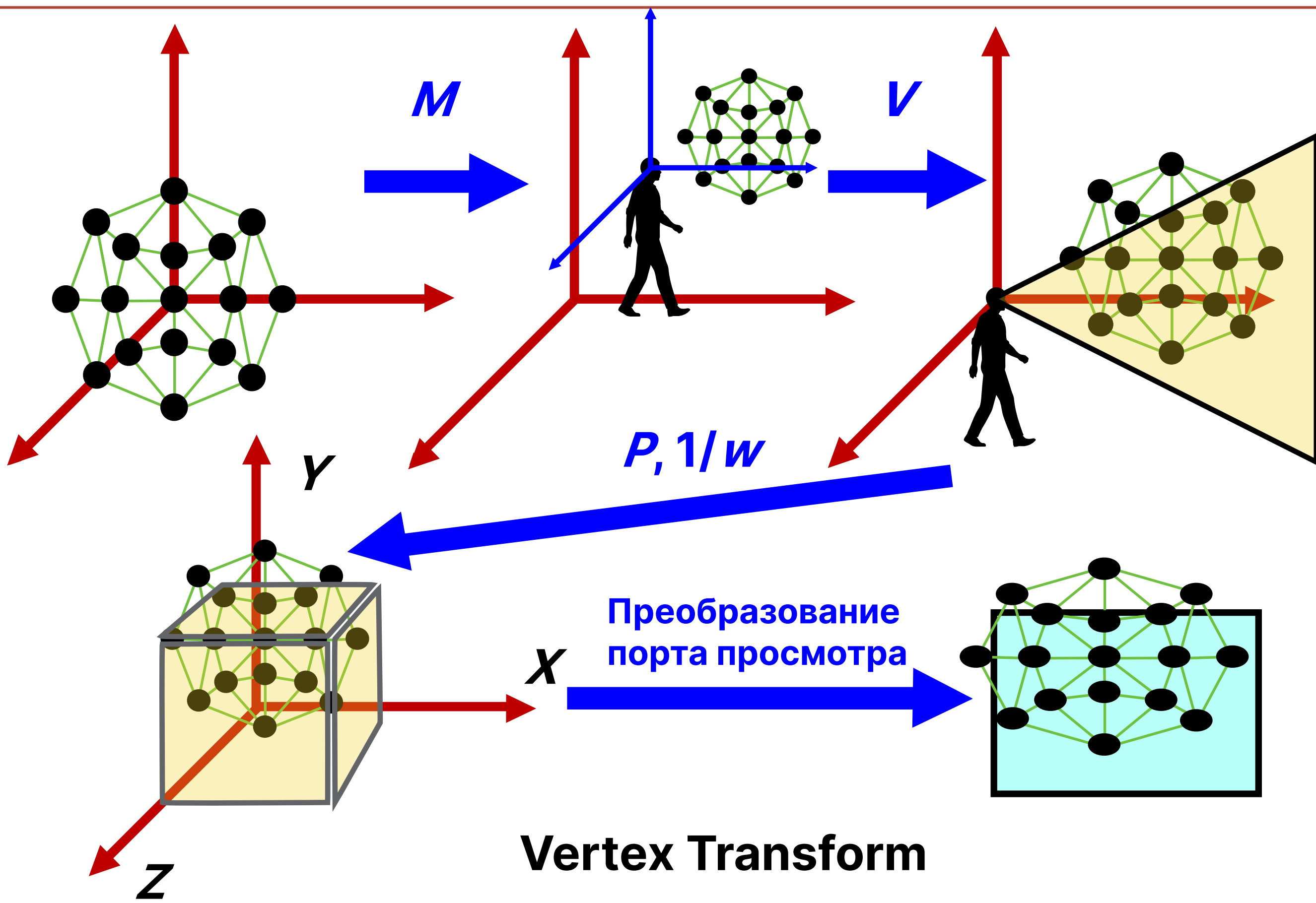
- **Сцена** состоит из **объектов**
- **Объекты** состоят из **примитивов**
- **Примитивы** состоят из **вершин**
- **Вершины** имеют **3D-координаты**
- Чтобы получить картинку, нужно:
 1. Разместить объекты в пространстве
 2. Спроецировать их примитивы на плоскость экрана
 3. Подобрать для проекций пиксели (**растеризация**)
 4. Раскрасить пиксели



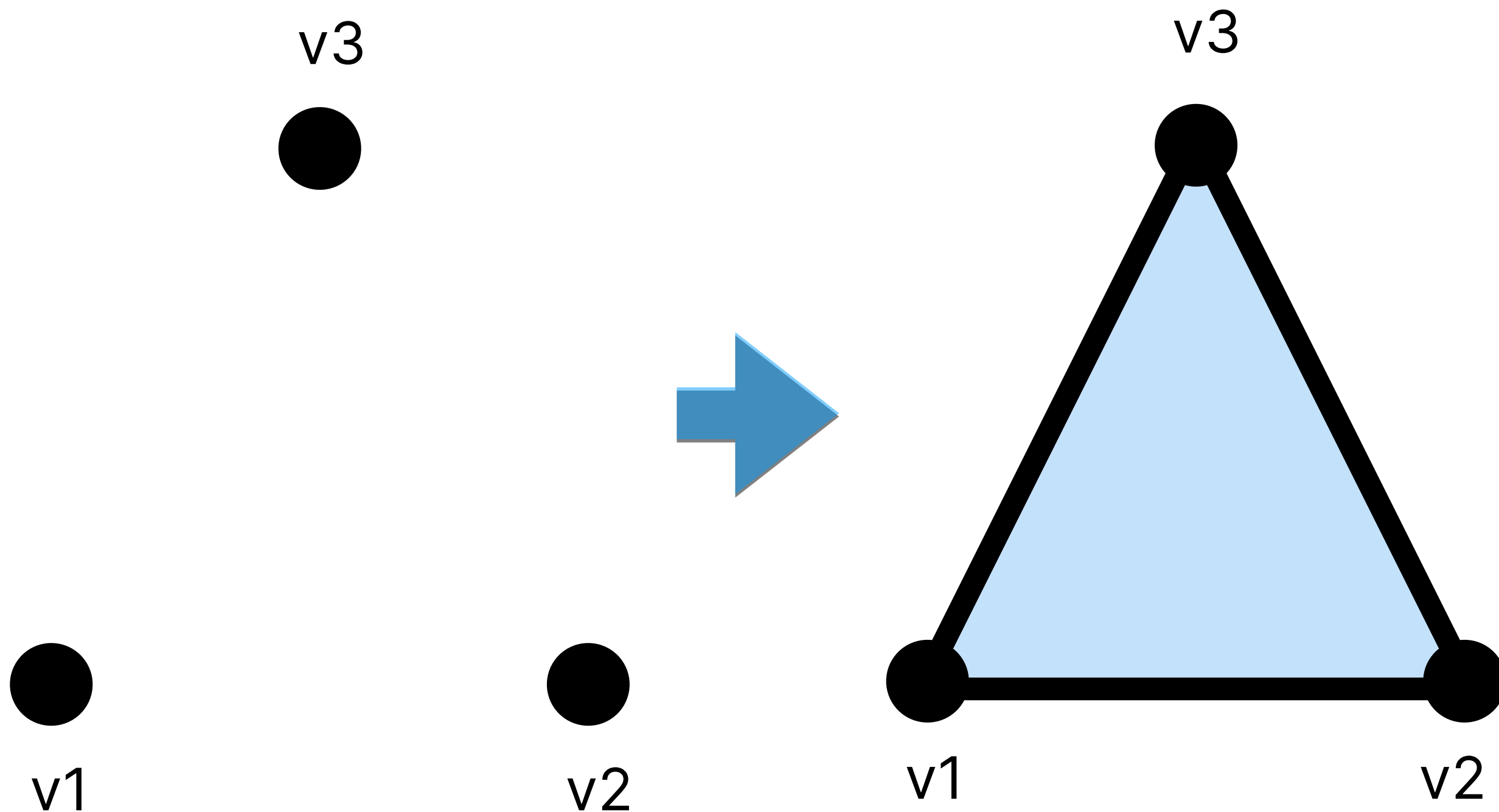
Draw Call



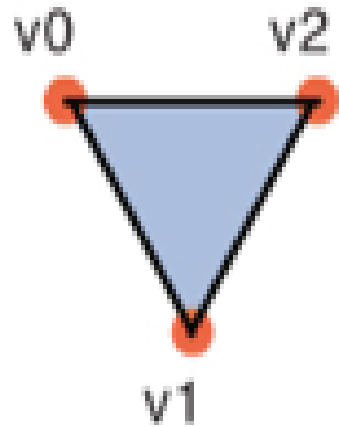




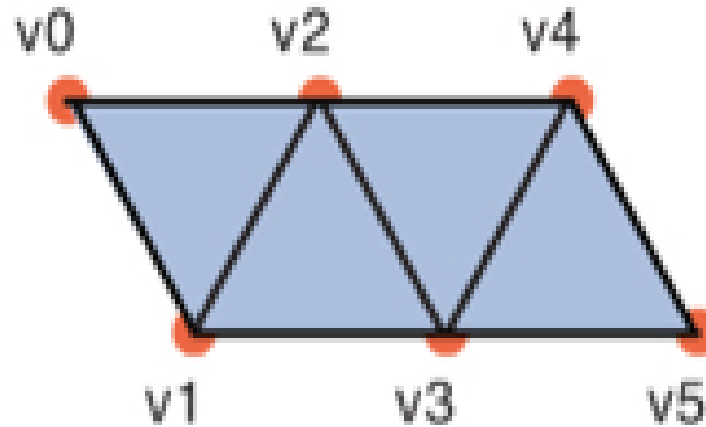
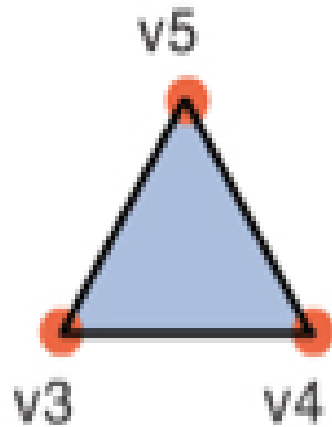
Primitive Assembly



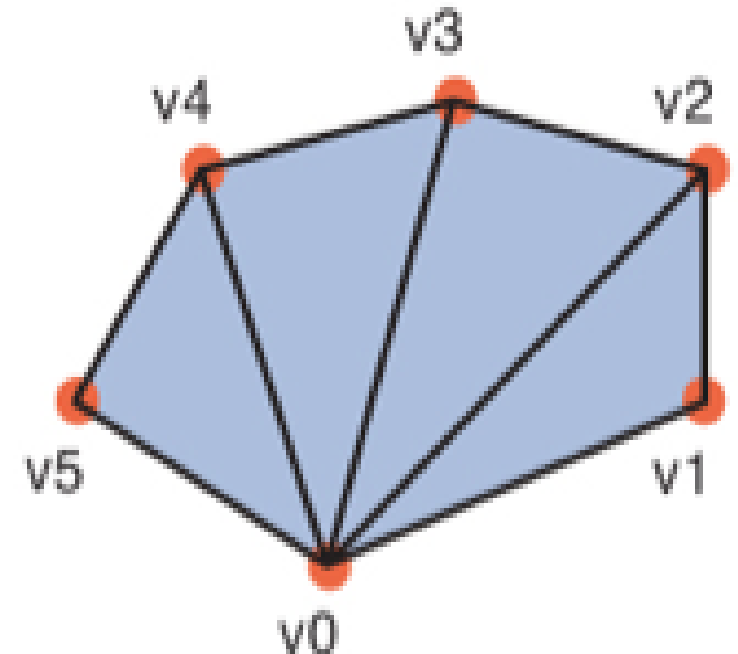
Правила сборки



**Треугольники
(Triangles)**

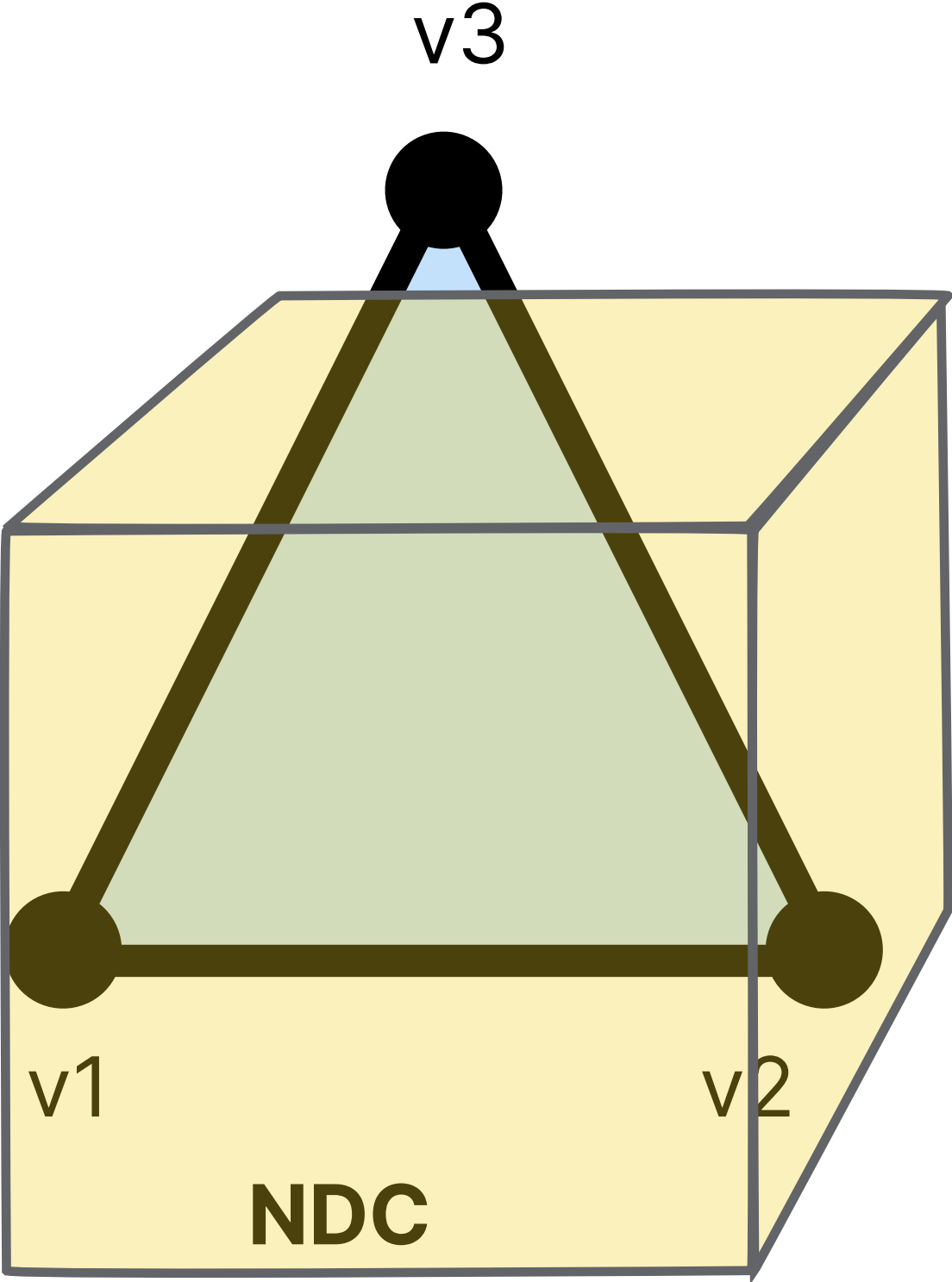
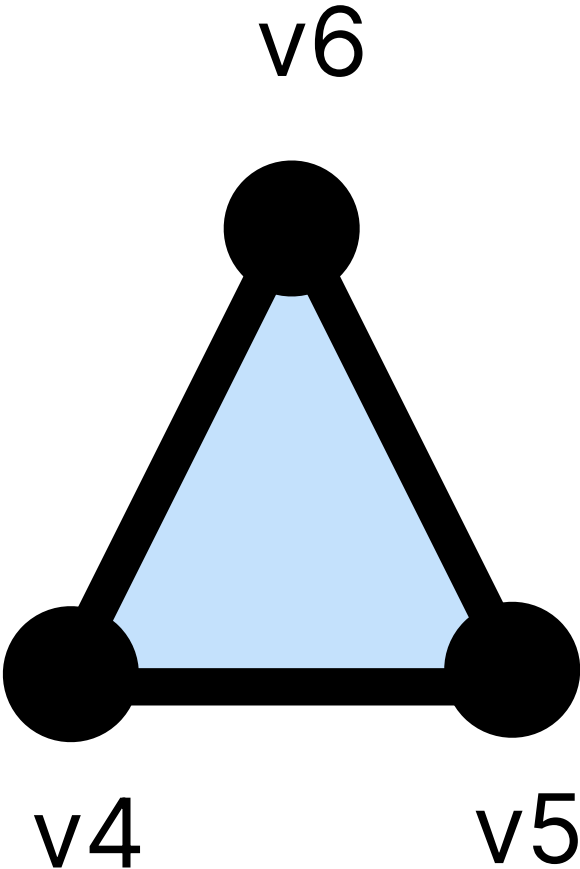


**Цепочка треугольников
(Triangle Strip)**

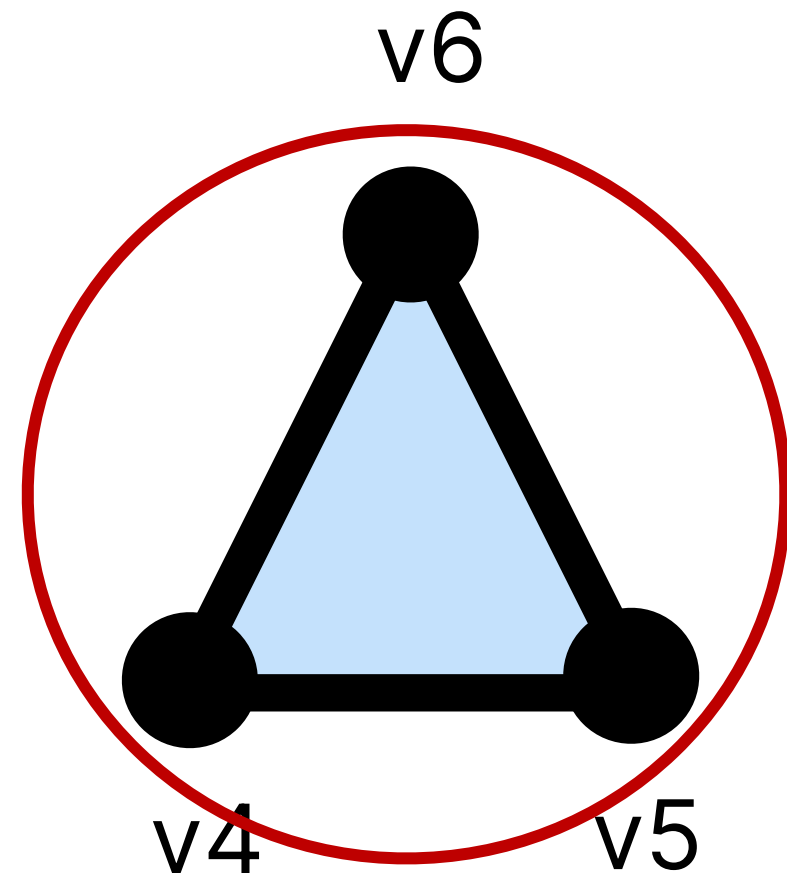


**Веер треугольников
(Triangle Fan)**

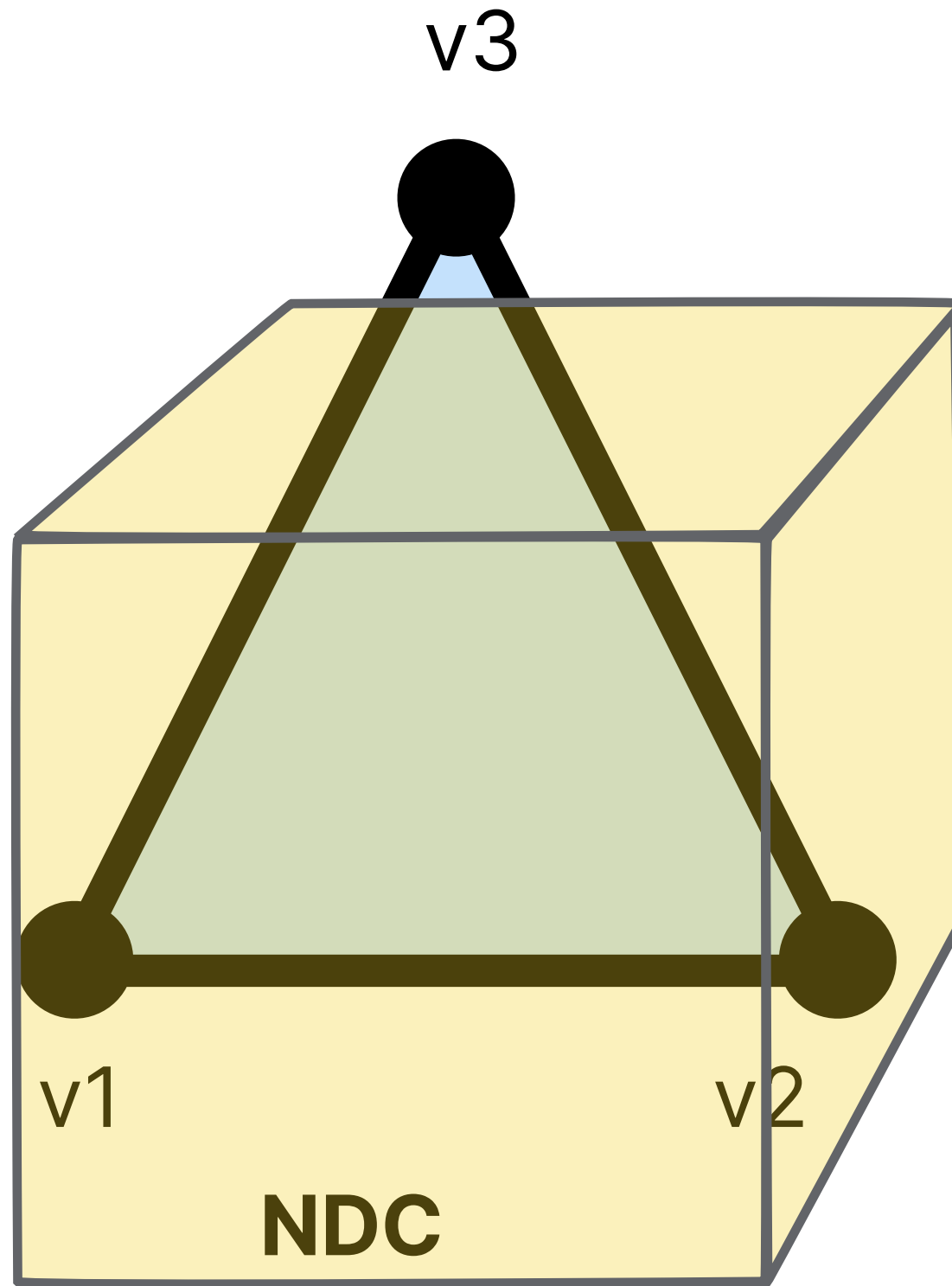
Clipping



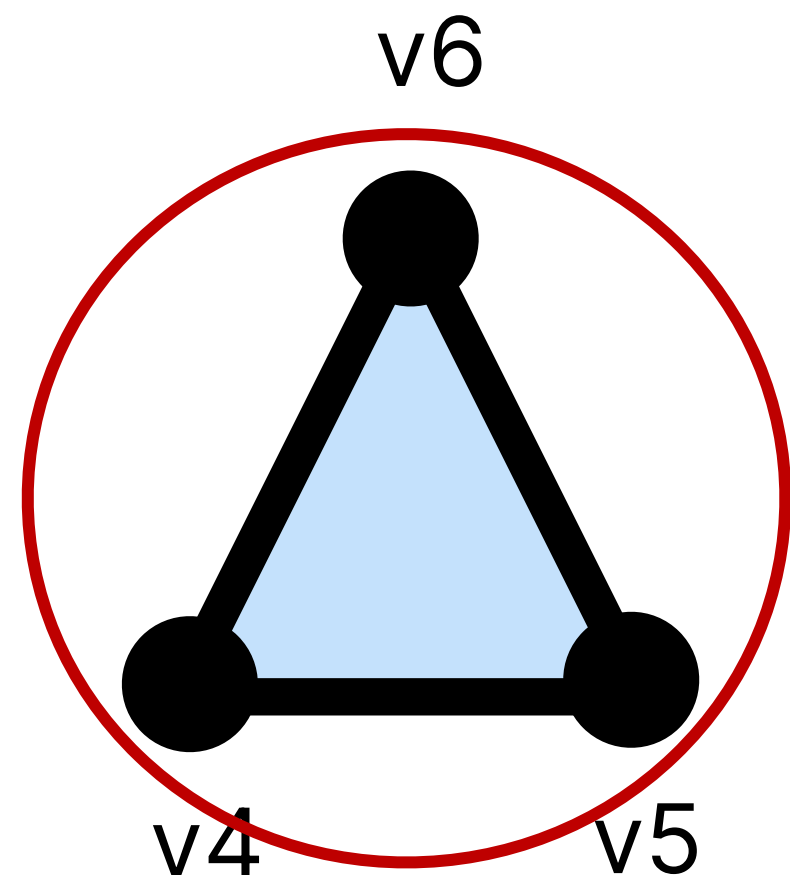
Clipping



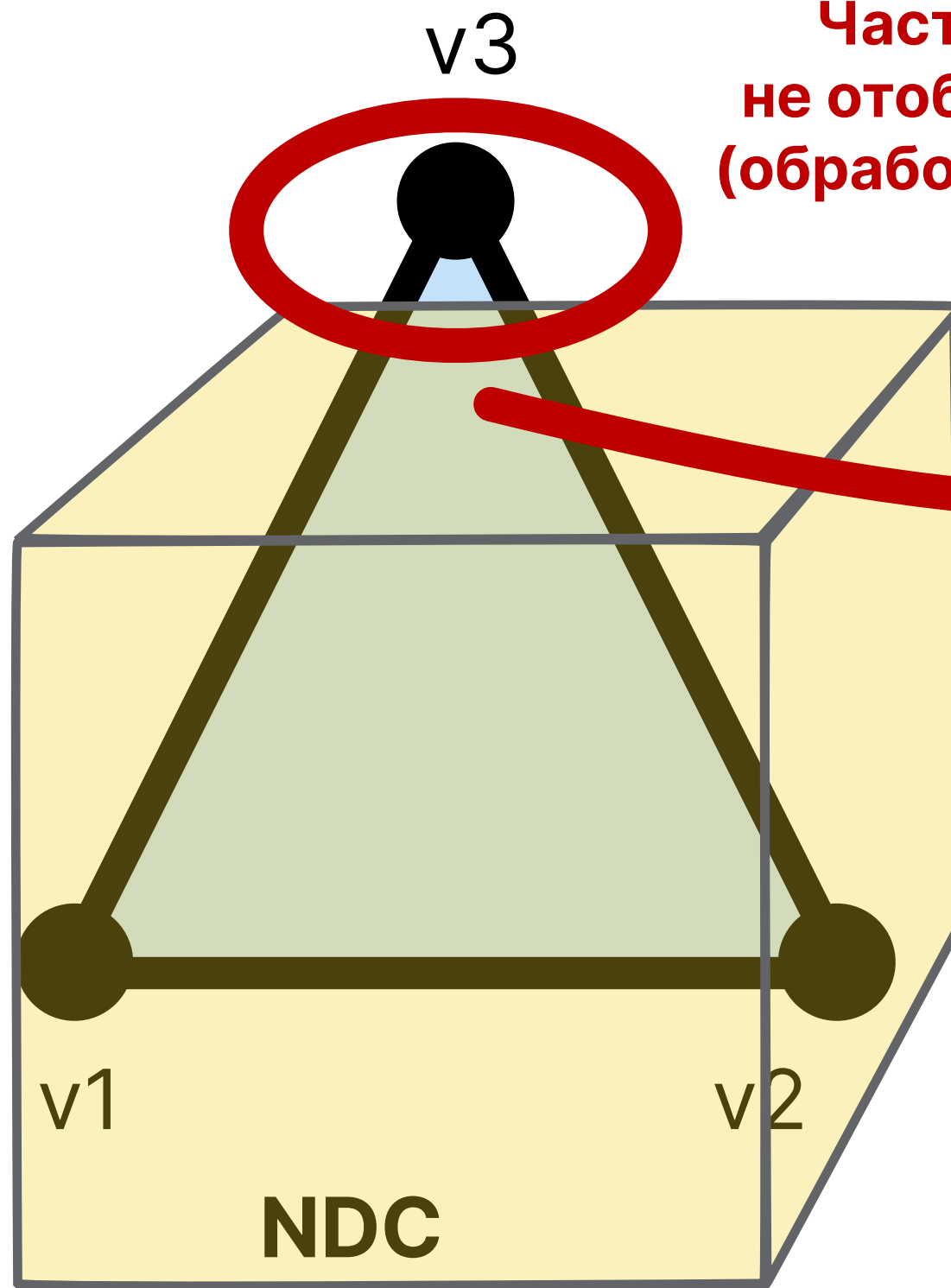
**Треугольник
полностью
не отобразится
на экране
(обработка
прекращается)**



Clipping

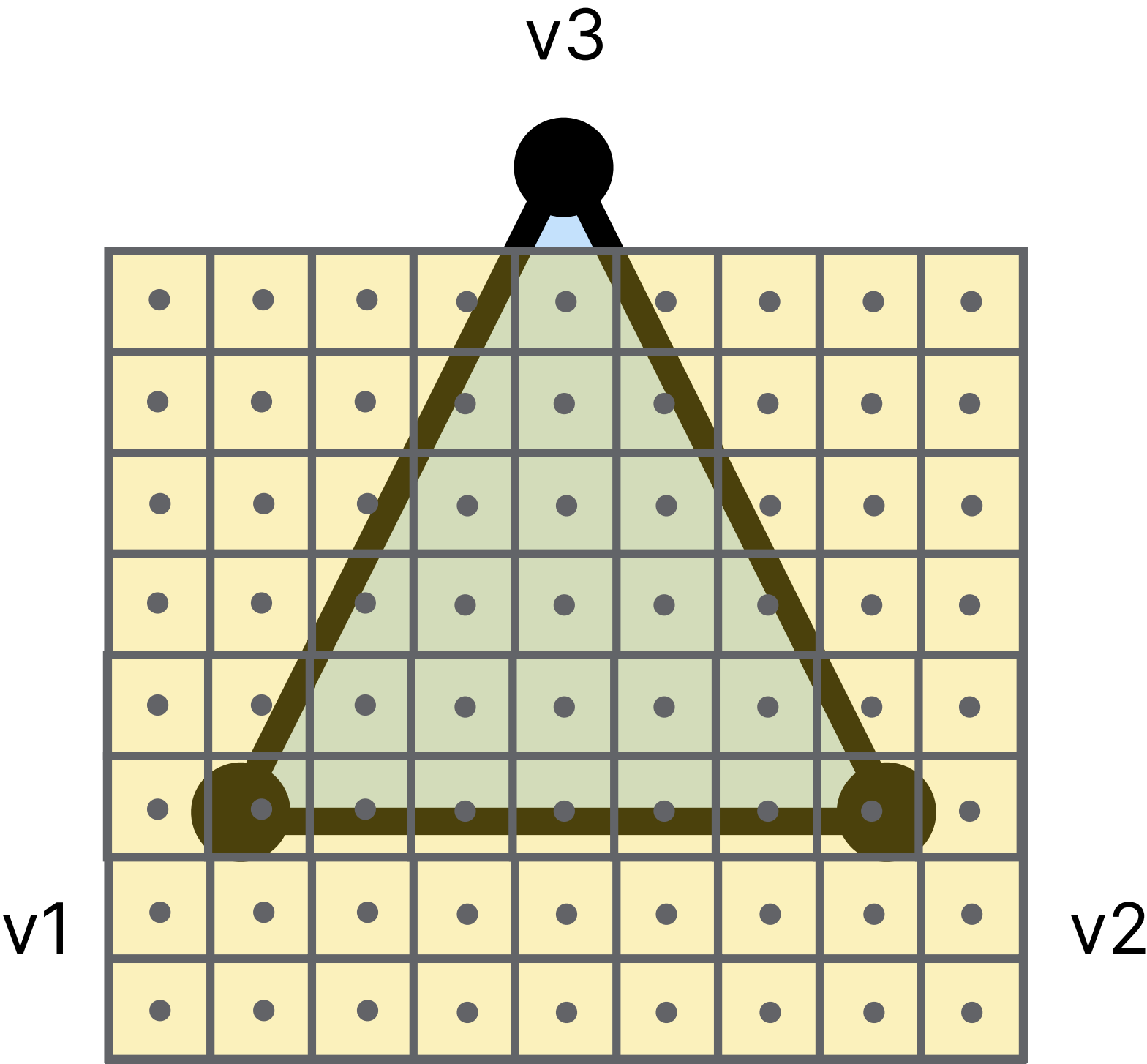


Треугольник
полностью
не отобразится
на экране
(обработка
прекращается)

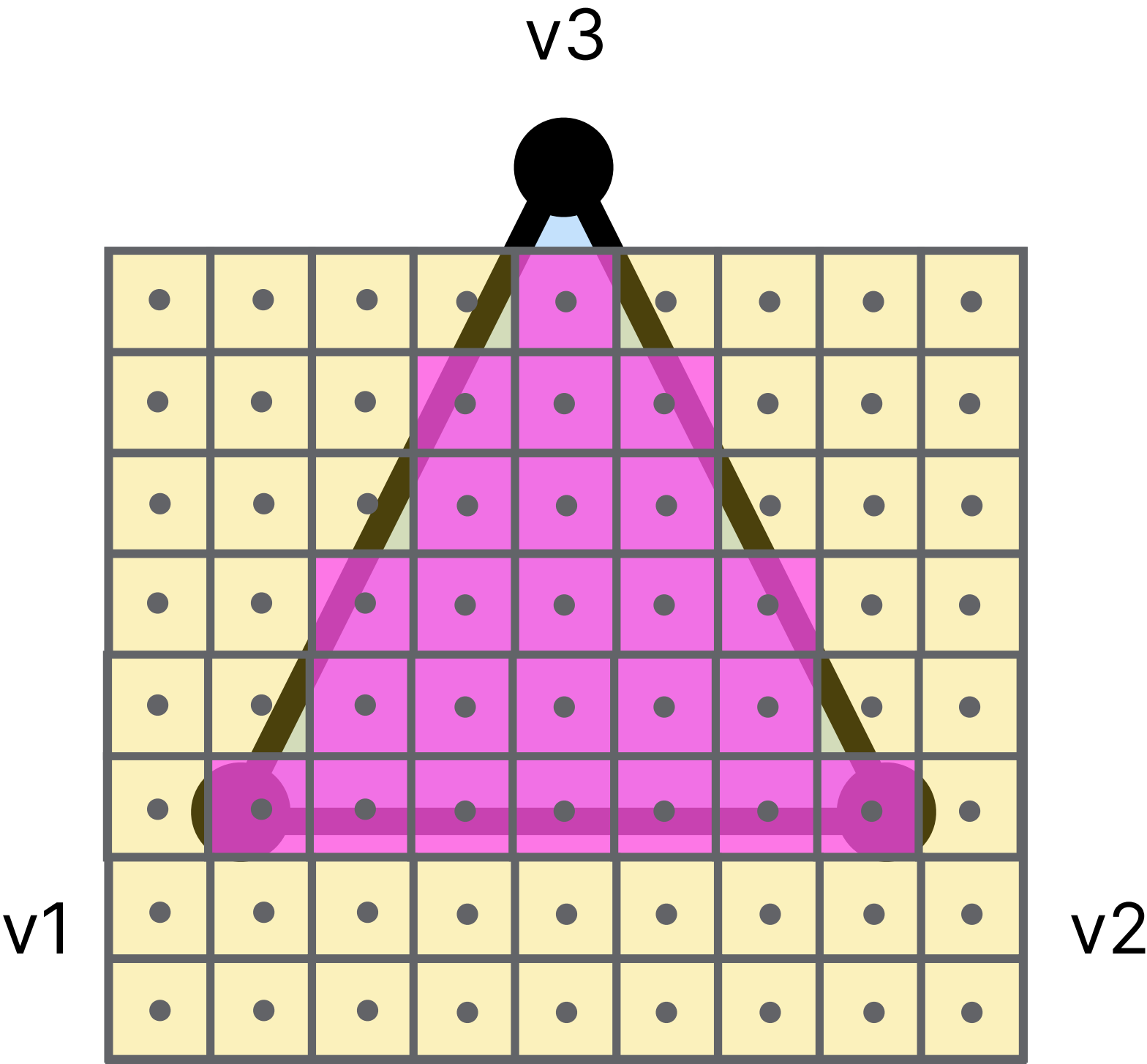


Часть треугольника
не отобразится на экране
(обработка продолжается)

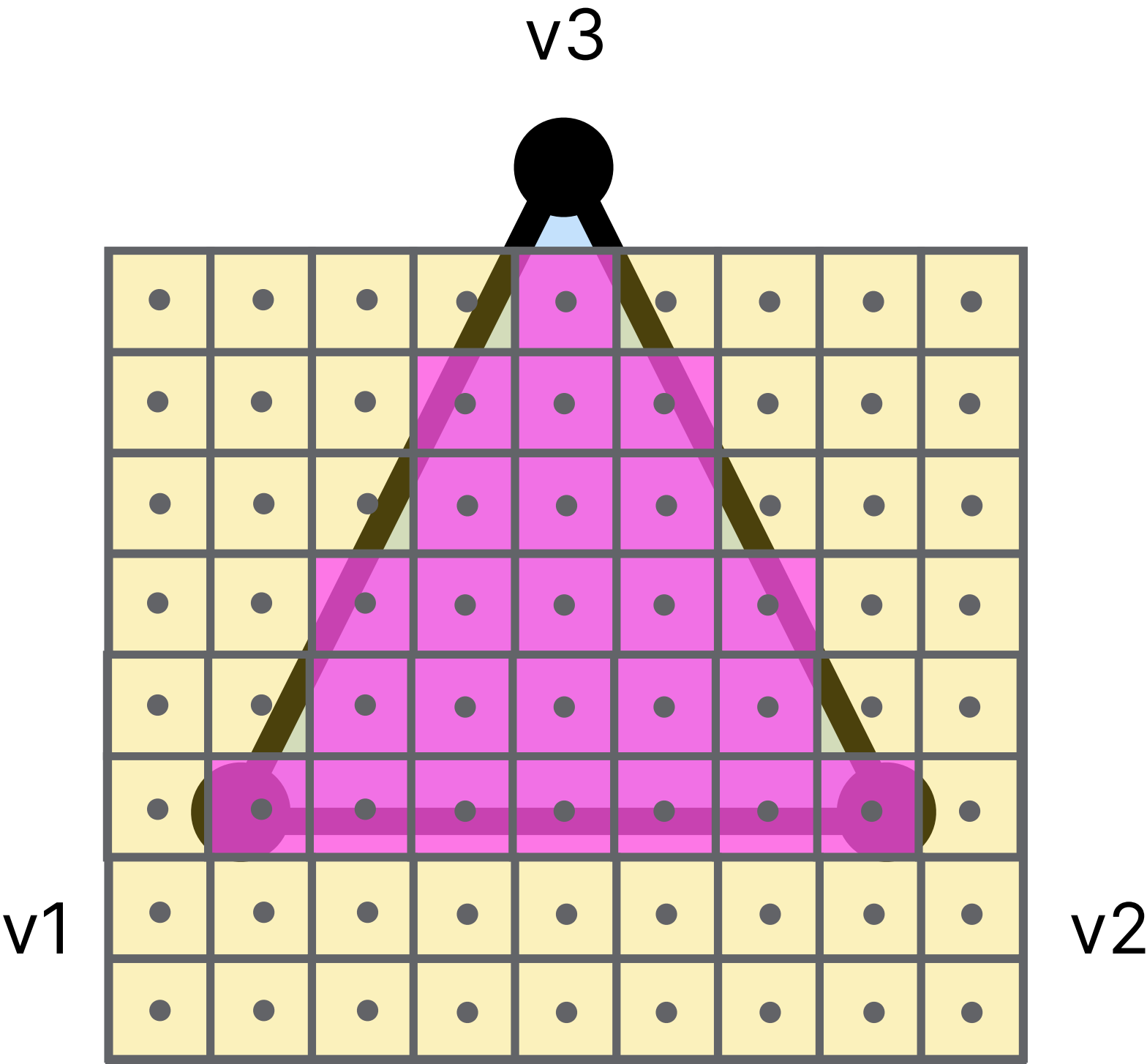
Rasterization



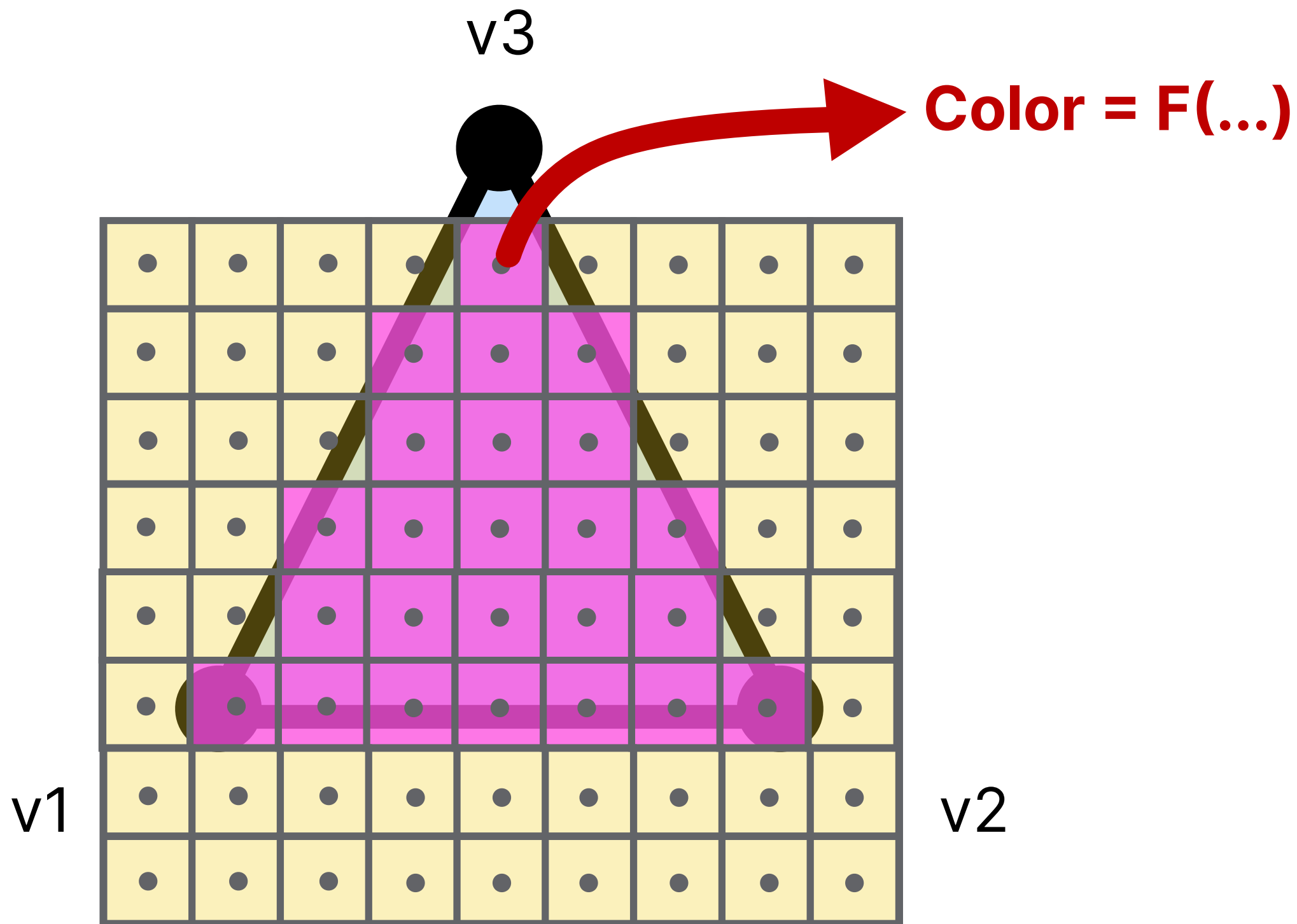
Rasterization



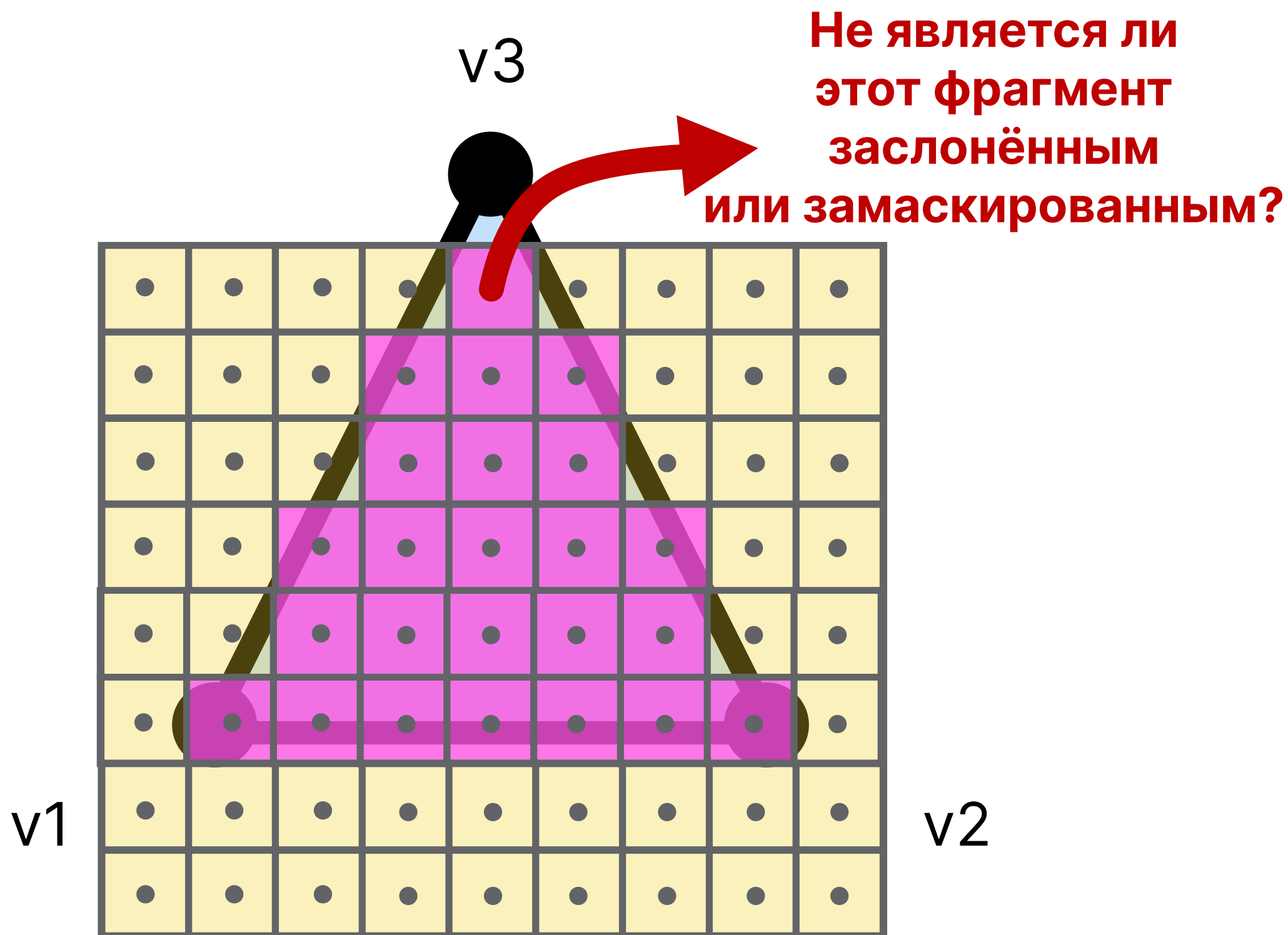
Fragment Transform



Fragment Transform



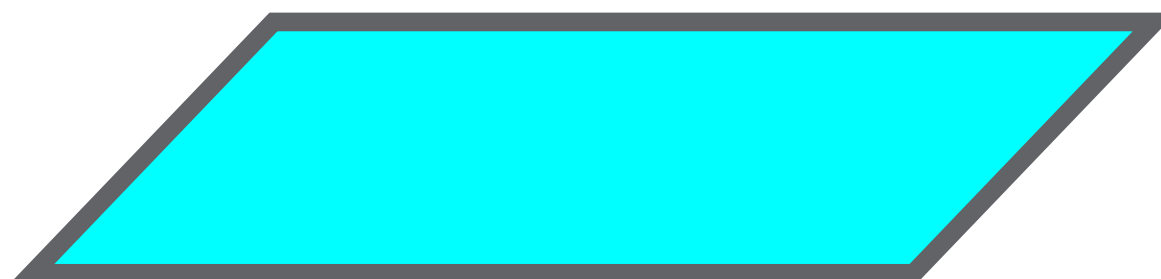
Visibility Test



Blending



Фрагмент

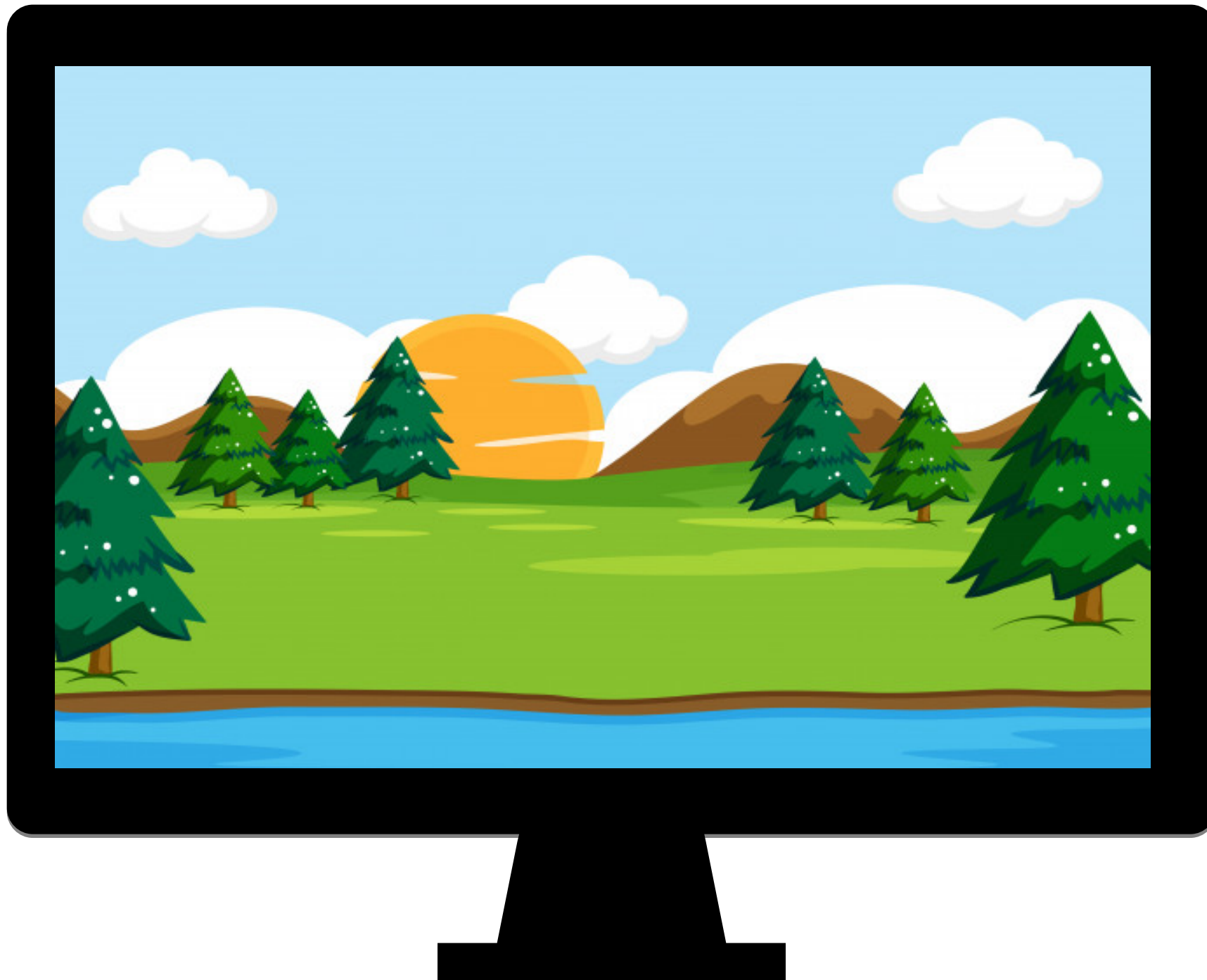


Фон



Результат

Presentation



Большая часть графических API имеют следующую концептуальную организацию данных для визуализации:

Графический контекст

Буфер кадра

Буфер цвета

Буфер глубины

Буфер трафарета

■ ■ ■

Буфер вершин

Буфер атрибутов

Шейдер

Текстура

Машина состояний

- Поверхность рендеринга – объект в составе отображающей (оконной) системы, при помощи которого происходит демонстрация результатов визуализации сцены
- Поверхность рендеринга неразрывна связана с графическим контекстом и является системно-зависимой
- Обычно поверхность рендеринга включает в себя механизм **двойной буферизации** для обеспечения атомарности обновления экрана
- Смена активного и видимого буферов обычно аппаратно синхронизируется с обновлением монитора (**вертикальная синхронизация**)

- **Буфер кадра** – структура данных (часть графического контекста), агрегирующая буферы, хранящие результаты работы графического конвейера
 - **Буфер (внутри буфера кадра)** – структура данных для хранения результатов работы графического конвейера
 - За некоторыми буферами внутри буфера кадра чётко закреплено их назначение, другие же используются для сохранения произвольной информации
 - **Буфер цвета** – буфер для хранения итогового изображения
 - Буферы произвольного назначения внутри буфера кадра называются целями рендеринга
- **Буфер вершин** – структура данных для хранения вершин (с их атрибутами) и их индексов
- **Буфер атрибутов** – структура данных для хранения настроек конвейера, связанных с вершинами

- 1. Инициализировать графический контекст и поверхность рендеринга**
- 2. Создать графические ресурсы**
- 3. Построить сцену**
- 4. Пока не дана команда завершения, повторять:**
 - 4.1. Изменить состояние сцены в соответствии с текущим состоянием приложения**
 - 4.2. Отправить данные на графический конвейер**
 - 4.3. Обновить поверхность рендеринга**
- 5. Удалить графические ресурсы**
- 6. Удалить графический контекст и поверхность рендеринга**

Анимация – это вдыхание жизни в сцену :)

Анимация – это последовательное отображение
(конструктивно) **кадров с различным содержанием**

Программно-аппаратная поддержка анимации:
механизм двойной буферизации

**Строго говоря, любое свойство объекта
может быть анимировано**

Важное требование к анимации:
сохранение межкадрового соответствия
(interframe coherence)

Анимация по таймеру:

- **Идея: запуск некоторого таймера (должен поддерживаться системой), который в наперёд заданные моменты времени вызывает функцию обновления состояния**
- **Способ имеет право на существование лишь в исключительных случаях**
- **Таймер лучше использовать как триггер анимации, а не как её «драйвер»**

Непрерывная анимация:

- **Идея: обновление состояния происходит с максимально возможной скоростью, а величина изменения вычисляется на основе желаемой и фактической скорости**

$$p^{(i)} = p^{i-1} + \Delta_p \Delta_t$$

p – значение анимируемого параметра

i – номер итерации графического цикла

Δ_p – желаемое изменение параметра p за секунду

Δ_t – время в секундах, прошедшее с момента
предыдущего изменения p